

TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE OAXACA

CARRERA

Ingeniería en Sistemas Computacionales

MATERIA

Programación Web

NOMBRE DE ACTIVIDAD O PROYECTO

DOCUMENTACIÓN DEL PROYECTO MARKITO

GRUPO:

7SA

ALUMNOS:

Castellanos Ramirez Marcos Jostin
Gijón Salgado Omar

PROFESOR:

Velazquez Hernandez Maricarmen Montserrat

OAXACA DE JUÁREZ, OAXACA, 26 DE MARZO DEL 2026

Resumen del Proyecto.

Descripción del proyecto: MarkITO es una aplicación web de comercio electrónico tipo C2C (Consumer to Consumer) diseñada exclusivamente para la comunidad estudiantil del Instituto Tecnológico de Oaxaca. Su propósito es centralizar, organizar y dar seguridad a la compra, venta e intercambio de materiales académicos de segunda mano (libros, calculadoras, batas, etc.) que suelen tener una vida útil corta dentro del plan de estudios.

Objetivo general: Desarrollar e implementar una aplicación web de comercio electrónico tipo C2C exclusiva para la comunidad estudiantil, que facilite la compra, venta e intercambio de materiales académicos de segunda mano, validando la identidad de los usuarios mediante correo institucional para garantizar un entorno seguro y confiable.

Objetivos específicos:

- Diseñar una interfaz de usuario (UI) responsiva e intuitiva centrada en la experiencia móvil (Mobile-First).
- Implementar un sistema de autenticación que restrinja el acceso exclusivamente a usuarios con dominio @itoaxaca.edu.mx.
- Desarrollar un módulo de gestión de anuncios (CRUD) que permita subir productos con fotos, descripción, precio y categorías.
- Integrar un sistema de mensajería interna o chat básico.
- Incorporar un modelo de negocio Freemium mediante la funcionalidad de "Anuncios Destacados".

Alcances [Funcionalidades]:

- Formulario de registro y login de usuario
- Validación de registro (administrador)
- CRUD de las publicaciones (usuario y administrador)
- CRUD de los reportes (el usuario los crea y el administrador los recibe)
- Validación de publicaciones por el administrador
- Creación de categorías por parte del administrador
- CRUD de comentarios por parte de los usuarios
- Administración general de usuarios

Detalles del Proyecto.

Lista de requerimientos funcionales:

- **RF-01 Registro y Login:** Permitir registro solo con correos del dominio estudiantil y enviar link de verificación.
- **RF-02 Gestión de Perfil:** Permitir al usuario editar su alias, foto y ver su historial de ventas/compras.
- **RF-03 Publicar Anuncio:** Crear anuncios con título, descripción, precio, estado, categoría y hasta 3 fotos.
- **RF-04 Búsqueda y Filtros:** Buscar por palabras clave y filtrar por categoría, precio y carrera.
- **RF-05 Destacar Anuncio:** Permitir pago de tarifa para que el anuncio aparezca al inicio de las listas.

- **RF-06 Chat/Contacto:** Habilitar un botón que abra un chat privado o revele el WhatsApp del vendedor.
- **RF-07 Reportes:** Permitir reportar anuncios inapropiados.
- **RF-08 Moderación:** Permitir al administrador eliminar anuncios reportados y banear usuarios.

Listado de requerimientos no funcionales:

- **RNF-01 Usabilidad:** Diseño responsivo y Mobile-First.
- **RNF-02 Rendimiento:** Tiempo de carga no mayor a 2 segundos en conexión 4G.
- **RNF-03 Disponibilidad:** Estar disponible el 99% del tiempo durante el periodo académico.
- **RNF-04 Seguridad:** Contraseñas encriptadas (hashing) y ocultamiento de correos sin consentimiento.
- **RNF-05 Escalabilidad:** Soportar al menos 1,000 productos activos simultáneamente.
- **RNF-06 Stack Tecnológico:** Uso de tecnologías web modernas para facilitar el mantenimiento.

Diagrama de Clase

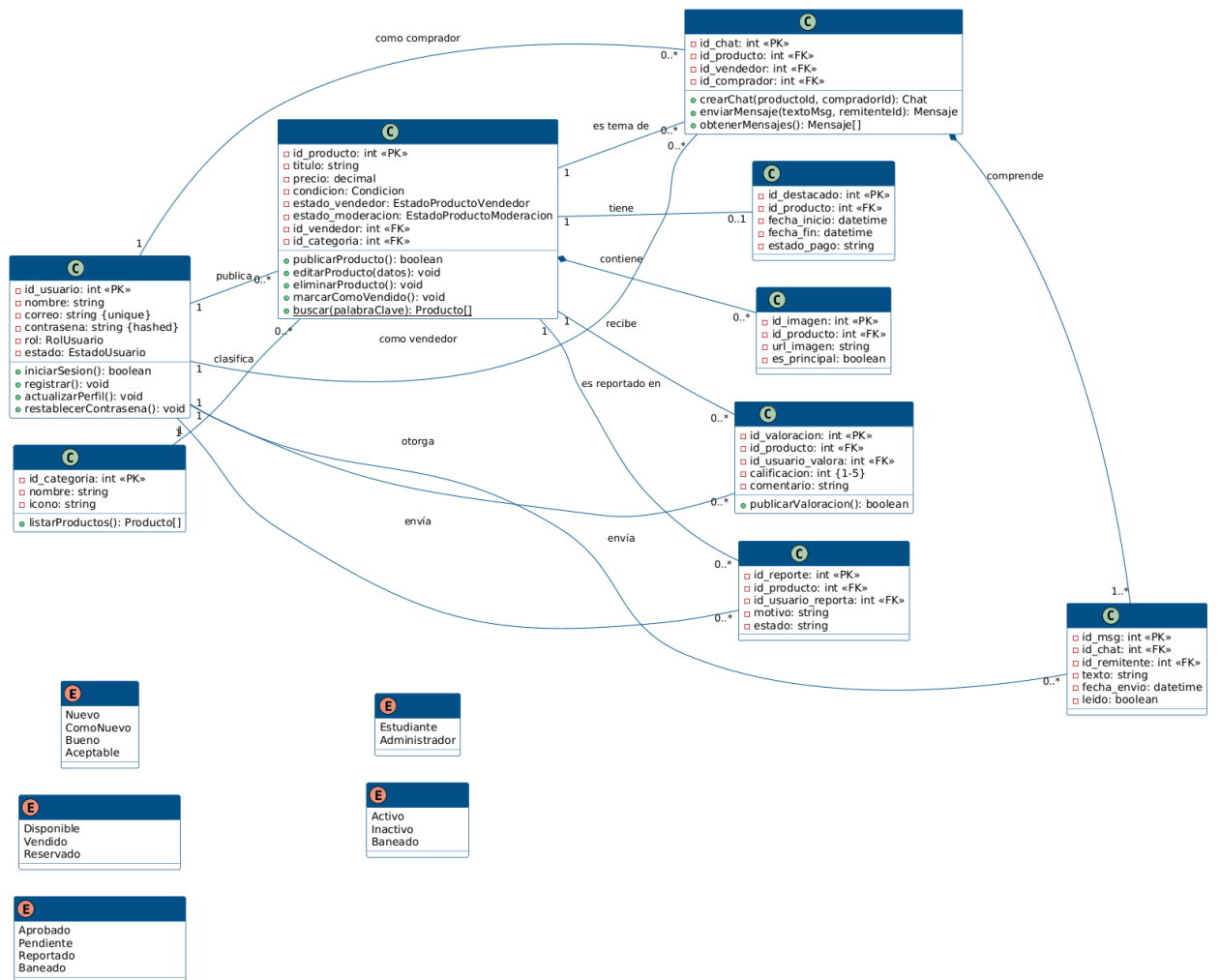


Diagrama Entidad-Relación

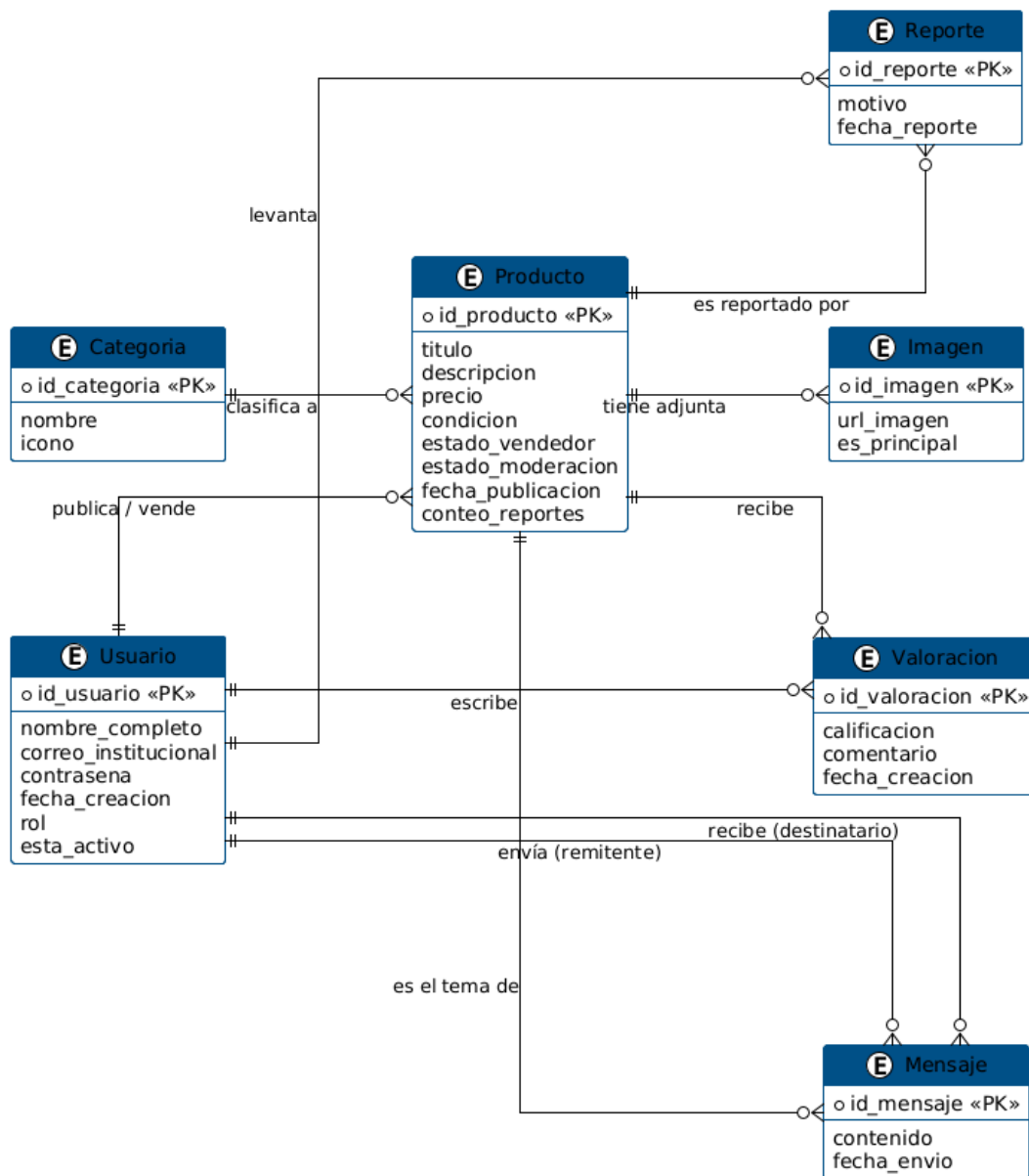
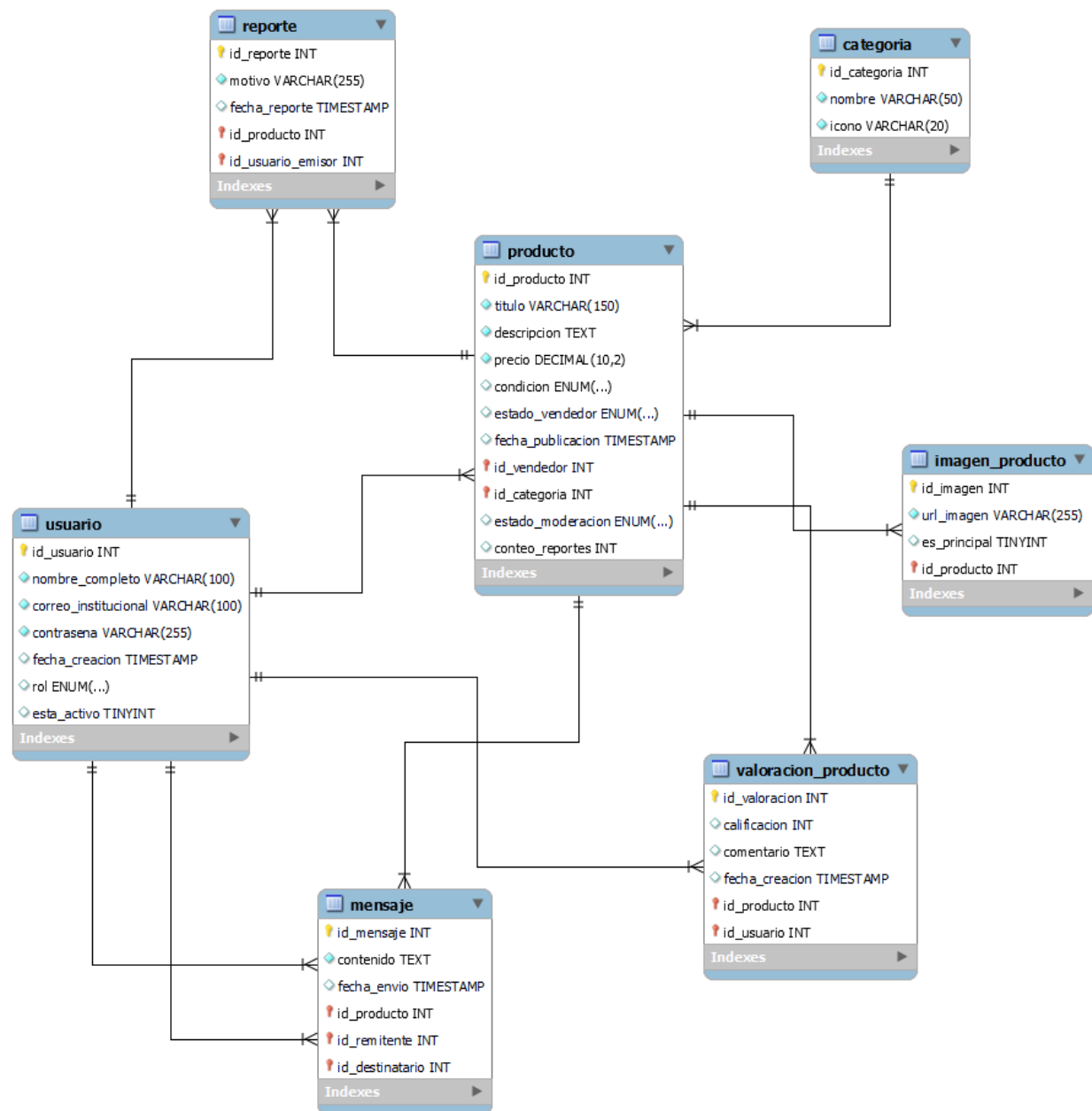


Diagrama Relacional.



Tecnologías.

Justificación de tecnologías a utilizar:

- **Frontend:** React y Tailwind CSS. Se eligen para garantizar que la plataforma web que utilizarán los alumnos sea rápida, moderna y amigable. Por medidas de seguridad, esta interfaz gráfica no tocará la base de datos directamente.
- **Backend:** PHP. Como está preconfigurado en XAMPP, funcionará como un "puente" lógico intermediario que recibe las peticiones desde React, consulta la base de datos de manera segura y le devuelve los datos correctos al usuario.
- **Base de Datos (BD):** MySQL. Se eligió por cumplir con el enfoque de un modelo relacional, aprovechando el servidor local XAMPP y MySQL Workbench para el diseño de tablas y relaciones seguras.

Anexos

Investigación de Servidores.

Cuadro Comparativo de Servidores Web

Característica	Apache HTTP Server	Nginx	Microsoft IIS	Caddy
Tipo de Licencia	Open Source (Apache License 2.0).	Open Source (BSD-like) y versión comercial (Nginx Plus).	Propietaria (Incluido en licencias de Windows Server).	Open Source (Apache License 2.0).

Sistema Operativo	Multiplataforma (Linux, Unix, Windows, macOS).	Multiplataforma (Optimizado para Linux/Unix; soporte Windows limitado).	Exclusivo de Windows.	Multiplataforma (Linux, Windows, macOS, Android).
Lenguajes Soportados	Soporte nativo mediante módulos (PHP, Python, Perl, etc.) dentro del mismo proceso.	No procesa lenguajes nativamente; usa FastCGI o proxy inverso (PHP-FPM, uWSGI, etc.).	Optimizado para .NET y ASP.NET. Soporta otros vía FastCGI.	Escrito en Go. Se integra con aplicaciones vía proxy inverso o FastCGI.
Rendimiento y Concurrencia	Arquitectura basada en procesos/hilos. El rendimiento decae con miles de conexiones simultáneas (bloqueo I/O).	Arquitectura asíncrona orientada a eventos. Extremadamente eficiente con alta concurrencia y bajo consumo de memoria.	Basado en hilos del sistema operativo. Buen rendimiento en entornos Windows, pero consume más recursos que Nginx.	Asíncrono (Go scheduler). Excelente rendimiento, soporta HTTP/3 por defecto (muy rápido).
Facilidad de Configuración	Media. Usa archivos de texto (httpd.conf) y	Media/Alta. Sintaxis lógica pero estricta. No permite	Alta (Visual). Interfaz gráfica potente, aunque	Muy Alta. Archivo Caddyfile muy legible y

	permite configuración descentralizada por carpeta (.htaccess).	configuración por carpeta (no existe .htaccess).	permite configuración vía XML (web.config) y PowerShell.	conciso. Configuración automática de muchas tareas.
Seguridad	Muy robusta y probada. Gran cantidad de módulos de seguridad (ej. ModSecurity).	Excelente. Arquitectura segura por diseño (aislamiento de procesos).	Integrada con Active Directory y autenticación de Windows.	Memoria segura (Go). HTTPS automático por defecto (gestiona certificados TLS solo).
Escalabilidad	Escala verticalmente bien, pero horizontalmente requiere configuración compleja.	Rey del escalado horizontal. Funciona excelente como balanceador de carga.	Escala bien en ecosistemas Microsoft Azure/Windows Server.	Escala muy bien en entornos de contenedores (Docker/Kubernetes).
Casos de Uso Recomendados	Hosting compartido (por el .htaccess), sitios dinámicos clásicos (LAMP stack),	Servidor de contenido estático, Proxy Inverso, Balanceador de carga, sitios de tráfico muy alto.	Entornos corporativos que usan tecnologías Microsoft (.NET, SharePoint, Exchange).	Microservicios, desarrolladores que buscan "Zero-config", sitios que requieren HTTP/3 y HTTPS rápido.

	entornos de desarrollo.			
--	-------------------------	--	--	--

1. ¿Qué es un servidor web y cuál es su función dentro de una arquitectura web?

Un servidor web es un sistema cuya función principal es almacenar, procesar y entregar contenido de sitios web a los usuarios.

Su función en la arquitectura:

- **Escucha:** Se mantiene a la espera de peticiones entrantes a través de protocolos estándar en puertos específicos.
- **Resolución:** Cuando recibe una petición, el servidor busca el recurso solicitado en su sistema de archivos.
- **Entrega:**
 - Si es **contenido estático**, lo envía directamente al navegador.
 - Si es **contenido dinámico**, delega el procesamiento a un intérprete o servidor de aplicaciones y devuelve el resultado final al usuario.

2. ¿Qué diferencias existen entre un servidor web y un servidor de aplicaciones?

- **Servidor Web:**
 - **Foco:** Manejo del protocolo HTTP/HTTPS.
 - **Especialidad:** Servir contenido estático (archivos) de forma extremadamente rápida.
 - **Lógica:** Generalmente no ejecuta lógica de negocio compleja; actúa como el "repcionista" que entrega información ya lista.
- **Servidor de Aplicaciones:**
 - **Foco:** Ejecución de lógica de negocio y generación dinámica de contenido.
 - **Especialidad:** Procesar transacciones, conectar con bases de datos, ejecutar algoritmos complejos.
 - **Lógica:** Es la "cocina" donde se preparan los datos antes de que el servidor web los sirva.

3. ¿Por qué el servidor web es un componente crítico en el rendimiento de una aplicación web?

El servidor web es la puerta de entrada de todas las peticiones. Si este componente falla o es ineficiente, toda la aplicación se degrada, sin importar qué tan bien programado esté el código "backend".

Lenguaje y programas del lado del cliente.

Los Lenguajes del Desarrollo Web (Lado del Cliente)

HTML (HyperText Markup Language):

- **Qué es:** Es el lenguaje de marcado estándar para la creación de páginas web. No es un lenguaje de programación *per se*, sino un sistema de etiquetas.
- **Su función:** Proporciona la **estructura y el contenido** de la página. Define dónde van los encabezados, los párrafos, las imágenes, los enlaces y los formularios.

CSS (Cascading Style Sheets):

- **Qué es:** Es un lenguaje de hojas de estilo utilizado para describir la presentación de un documento escrito en HTML.
- **Su función:** Se encarga del **diseño y la apariencia visual**. Controla los colores, las fuentes, los márgenes, las animaciones y cómo se adapta el contenido a diferentes tamaños de pantalla (diseño responsivo).

JavaScript (JS):

- **Qué es:** Es un lenguaje de programación de alto nivel, dinámico e interpretado que los navegadores web pueden ejecutar.
- **Su función:** Añade **interactividad y comportamiento dinámico** a la web. Permite que las cosas sucedan cuando el usuario hace clic en un botón, envía formularios sin recargar la página, crea animaciones complejas y se comunica con servidores en segundo plano (API).

TypeScript (TS):

- **Qué es:** Es un "superconjunto" (*superset*) de JavaScript desarrollado por Microsoft. Esto significa que cualquier código JavaScript válido también es código TypeScript, pero TS le añade características adicionales, principalmente el **tipado estático**.
- **Su función:** Ayuda a los desarrolladores a escribir código más robusto y a **detectar errores antes de ejecutar la aplicación** (en tiempo de compilación en lugar de en tiempo de ejecución). Al final, los navegadores no entienden TypeScript, por lo que el

código se "transpila" (se traduce) a JavaScript normal para que la web funcione.

Cuadro Comparativo: Tecnologías del Lado del Cliente

Lenguaje	¿Qué función cumple?	¿En qué parte del navegador se ejecuta?	¿Cómo interactúan entre sí?
HTML	Define la estructura y el contenido (textos, imágenes, enlaces, áreas de la página).	Lo procesa el Motor de Renderizado (ej. Blink en Chrome) para construir el DOM (<i>Document Object Model</i>), que es el esqueleto interno de la página.	Es el punto de partida. Recibe las reglas visuales de CSS y expone sus elementos para que JavaScript los pueda leer o modificar.
CSS	Controla la presentación visual y el diseño (colores, fuentes, posiciones, adaptabilidad a móviles).	Lo procesa el Motor de Renderizado para construir el CSSOM (<i>CSS Object Model</i>). Luego, lo combina con el DOM para "pintar" la pantalla.	Se "engancha" a los elementos HTML a través de selectores (clases, IDs o etiquetas) para cambiar su aspecto sin alterar la estructura base.

JavaScript	Aporta interactividad, lógica y dinamismo (animaciones, validaciones, peticiones a servidores, eventos de clic).	Se ejecuta en el Motor de JavaScript del navegador (ej. el motor V8 en Google Chrome o SpiderMonkey en Firefox).	Es el "director de orquesta". Puede acceder al DOM (HTML) para agregar o borrar elementos, y al CSSOM (CSS) para cambiar estilos visuales en tiempo real según lo que haga el usuario.
TypeScript	Proporciona tipado estático y reglas estrictas a JavaScript para evitar errores durante el desarrollo y facilitar proyectos grandes.	No se ejecuta en el navegador. Los navegadores no entienden TypeScript. Se debe <i>transpilar</i> (traducir) a JavaScript normal antes de enviarlo al navegador.	Durante el desarrollo, actúa como una capa de seguridad sobre JavaScript. Una vez convertido a JavaScript, interactúa con HTML y CSS exactamente igual que el JS tradicional.

Frameworks del lado del cliente

¿Qué es un Framework (Entorno de Trabajo)?

En el desarrollo web del lado del cliente, un **framework** es un conjunto de herramientas, librerías y convenciones preestablecidas que proporcionan una base (un esqueleto o arquitectura) sobre la cual puedes construir tu aplicación web.

Cuadro Comparativo: Principales Frameworks del Lado del Cliente

Framework / Tecnología	Características Principales	Ventajas	Desventajas
Angular	Creado y mantenido por Google . Basado sólidamente en TypeScript . Usa una arquitectura muy estructurada (MVC). Es un framework "todo en uno" (trae sus propias herramientas para rutas, formularios, etc.).	- Ideal para aplicaciones empresariales gigantes y complejas. - Al ser estricto, todos los proyectos Angular se parecen, facilitando el trabajo en equipo. - Código muy robusto y predecible.	- Su curva de aprendizaje es muy pronunciada (es difícil para principiantes). - Puede ser excesivamente complejo y pesado para proyectos pequeños.

React	Creado y mantenido por Meta (Facebook) . Usa un concepto llamado Virtual DOM para actualizar la pantalla rapidísimo. Utiliza JSX (una mezcla de JavaScript y HTML en el mismo archivo).	- Es la tecnología más demandada en el mercado laboral. - Ecosistema gigante; hay librerías para resolver casi cualquier problema. - Muy flexible, te permite armar tu propia arquitectura.	- Como no es un framework completo, debes elegir y conectar muchas herramientas de terceros (lo que puede causar "fatiga de decisiones"). - Requiere aprender la sintaxis JSX.
Vue.js	Creado por un ex-ingeniero de Google (Evan You). Es un framework progresivo (puedes adoptarlo poco a poco en un proyecto existente). Separa claramente el HTML, CSS y JS en un solo archivo (Single File Components).	- Curva de aprendizaje muy suave (es fácil empezar si ya sabes HTML/CSS/JS). - Documentación excelente y clara. - Toma lo mejor de Angular (estructura) y de React (Virtual DOM y flexibilidad).	- Menor respaldo corporativo directo (aunque es sostenido por donaciones y una gran comunidad). - Mercado laboral ligeramente menor comparado con React y Angular.

Svelte	Es el "chico nuevo" revolucionario. A diferencia de los demás, no usa Virtual DOM. Actúa como un <i>compilador</i>: traduce tu código Svelte a código JavaScript puro y súper optimizado durante la construcción de la app.	- Las aplicaciones son extremadamente rápidas y los archivos finales son diminutos. - Escribes mucho menos código; su sintaxis es muy natural y limpia. - Excelente experiencia para el desarrollador.	- Al ser más nuevo, su comunidad es más pequeña. - Hay menos librerías de terceros, componentes pre-hechos o respuestas en foros en comparación con React.
---------------	--	--	---

Cuadro comparativo de lenguajes del lado del servidor

Lenguaje / Entorno	Rendimiento y Escalabilidad	Casos de Uso Ideales	Principales Ventajas	Principales Desventajas	Frameworks Populares
PHP	Medio. Escala bien, pero consume más recursos por cada petición.	E-commerce, CMS (WordPress), sitios web tradicionales, foros.	Despliegue baratísimo (funciona en cualquier hosting), enorme comunidad, fácil de integrar con HTML.	Código puede volverse desordenado si no se usan frameworks. Ejecución síncrona.	Laravel, Symfony, CodeIgniter
Node.js (JavaScript)	Alto. Excelente para manejar miles de conexiones simultáneas.	Aplicaciones en tiempo real (Chats), APIs REST rápidas, Streaming.	Permite usar el mismo lenguaje en Frontend (React) y Backend. Muy rápido gracias a su modelo asíncrono.	No es ideal para tareas que requieran mucho procesamiento o de CPU pesado (cálculos matemáticos).	Express.js, NestJS
Python	Medio/Bajo. No es el más rápido en ejecución pura.	Machine Learning, Análisis de datos, automatización, MVPs de startups.	Sintaxis extremadamente limpia y legible. Excelente para desarrollo rápido.	Velocidad de ejecución más lenta comparada con lenguajes compilados.	Django, Flask, FastAPI

Java	Muy Alto. Excelente manejo de memoria y subprocesos.	Sistemas bancarios, aplicaciones empresariales a gran escala, Android.	Altamente seguro, robusto y escalable. Orientación a objetos muy estricta y segura.	Mucha configuración inicial. Código muy largo (verboso) para tareas sencillas.	Spring Boot
C# (.NET)	Muy Alto. Rendimiento a la par o superior a Java.	Software empresarial, videojuegos (Unity), ecosistema Microsoft.	Rendimiento excepcional, muy seguro, herramientas de desarrollo increíbles (Visual Studio).	Tradicionalmente ligado a Windows (aunque ahora .NET Core funciona en cualquier lado). Curva alta.	ASP.NET Core
Ruby	Medio/Bajo. Similar a Python en velocidad.	Startups rápidas, plataformas web, desarrollo ágil.	La velocidad para crear un proyecto desde cero es insuperable ("Convención sobre configuración").	Rendimiento menor en aplicaciones masivas. Comunidad menos activa que JS o Python hoy en día.	Ruby on Rails